

Einführung in die Programmierung mit Python

HSLU Hochschule
Luzern

Syllabus

1. Python installieren; Jupyter Notebooks
2. Grundlagen: Arithmetik, Variablen und Datentypen
3. Methoden; Listen; Bedingte Anweisungen
4. Schleifen
5. **Benutzerdefinierte Funktionen**
6. Datenaufbereitung und Grafische Darstellungen

Funktionen

- Funktionen sind eine Möglichkeit, Code zu organisieren. Die Grundidee ist, dass Sie ein Stück Code, das wahrscheinlich mehr als einmal verwendet wird, in eine Funktion auslagern, damit es wiederverwendet werden kann.
- Wir haben bereits einige Funktionen kennengelernt, wie z.B. `print`, `sum`, etc.

```
In [ ]: sum([1, 2, 3])
```

- Eine Funktion akzeptiert null oder mehr Eingaben, sogenannte *Argumente*. `sum` oben nimmt (mindestens) eines (probieren Sie aus, was passiert, wenn Sie keine Argumente übergeben).

- Wenn ein Benutzer die Funktion aufruft, macht berechnet sie etwas aus ihren Argumenten und übergibt (meistens) das Ergebnis als *Rückgabewert*.
- Dieser Wert kann in einer Variablen gespeichert werden, um ihn später zu verwenden:

```
In [ ]: a = sum([1, 2, 3])
```

```
In [ ]: print(a)
```

Funktionen definieren

- Benutzerdefinierte Funktionen werden mit dem Schlüsselwort `def` deklariert:

```
In [ ]: def mypower(x, y): # null oder mehr Argumente, hier zwei  
        return x**y
```

```
In [ ]: b = mypower(2, 3)  
        print(b)
```

Beachten Sie, wie die vom Benutzer übergebenen Argumente innerhalb der Funktion als die Variablen `x` und `y` verfügbar sind.

Übung

Schreiben Sie eine Funktion `area`, die zwei Zahlen als Eingabe nimmt (die die Seitenlängen eines Rechtecks darstellen) und die Fläche des Rechtecks zurückgibt.

In []:

Mehrere Ausgaben

- Funktionen können mehr als ein Ausgabeargument haben:

```
In [ ]: def plusminus(a, b):  
        return a+b, a-b
```

```
In [ ]: c, d = plusminus(1, 2);  
        print(c, d)
```


Schlüsselwortargumente

- Einige Funktionen akzeptieren neben positionalen Argumenten* auch *Schlüsselwortargumente*, die per Namen übergeben werden:

```
In [ ]: help(print)
```

```
In [ ]: print(1, 2)  
print(1, 2, sep=', ')
```

Defaultargumente

- Es ist möglich, bei der Definition einer Funktion einen Defaultwert anzugeben, den die Funktion verwendet, wenn der Nutzer keinen Wert angibt:

```
In [ ]: def mypower(x, y=2): # Defaultargumente müssen am Ende stehen
        return x**y
mypower(3)
```

```
In [ ]: mypower(3, 3)
```

Übung, Fortsetzung

Schreiben Sie eine Funktion `area`, die zwei Zahlen als Eingabe nimmt (die die Seitenlängen eines Rechtecks darstellen) und die Fläche des Rechtecks zurückgibt. Wenn die zweite Eingabe nicht angegeben wird, soll die Funktion die Fläche eines Quadrats mit der Seitenlänge der ersten Eingabe berechnen.

Hinweis: Lassen Sie die zweite Eingabe standardmässig auf `None` gesetzt.

Erwartete Ausgabe:

```
area(2, 3) # sollte 6 zurückgeben  
area(2)    # sollte 4 zurückgeben
```

In []:

Zusammenfassung / weiterführende Literatur (optional)

- Optionales Notebook "Mehr über Funktionen"
- https://www.w3schools.com/python/python_functions.asp
- <https://python-course.eu/python-tutorial/functions.php>

Hausaufgabe

- Anfängerübung 16 von <https://holypython.com/beginner-python-exercises/>
- Übungen 2, 3, 6, 9 (schwierig), 10, 16 von <https://www.w3resource.com/python-exercises/python-functions-exercises.php>

